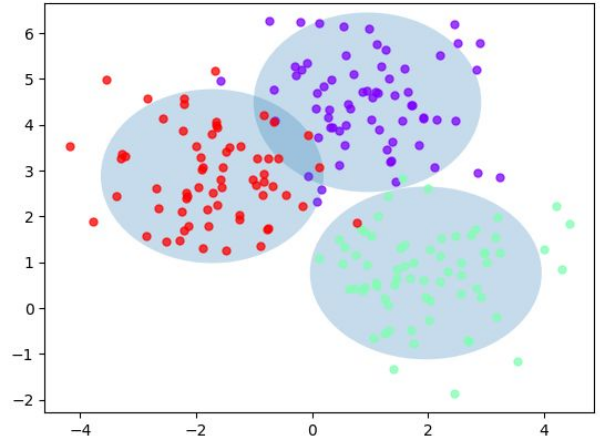
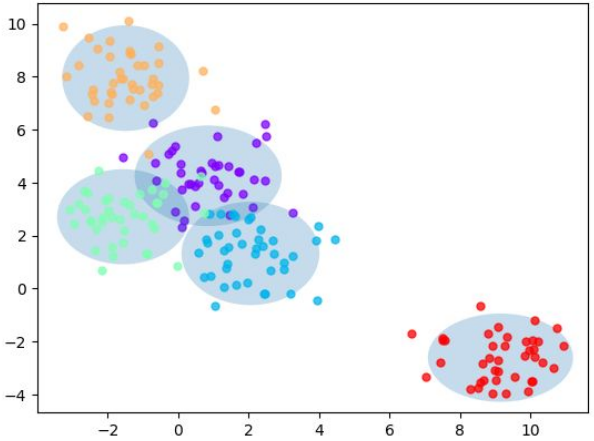
Lab 3

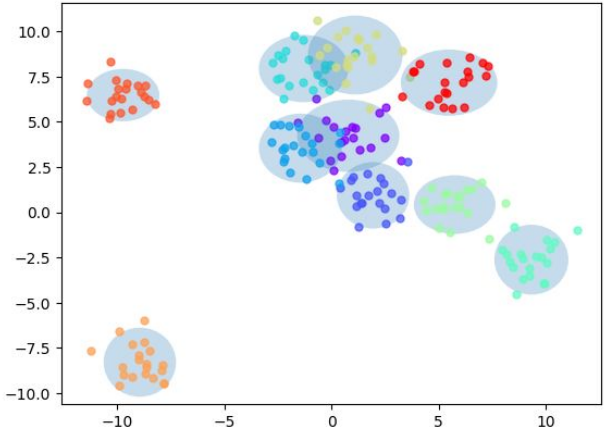
Matteo Camellini, Riccardo Fragale

Assignment 1: ML estimates for test data

- We wrote the mlParams function to compute the maximum likelihood (ML) estimates for the mean and covariance for each class in the dataset.
- We used the genBlobs() function to generate synthetic, Gaussian-distributed data points alongside their class labels to test our implementation.
- Finally, we computed the ML-estimates for this synthetic data and visualized the results by plotting the 95% confidence intervals using the provided plotGaussian() function.

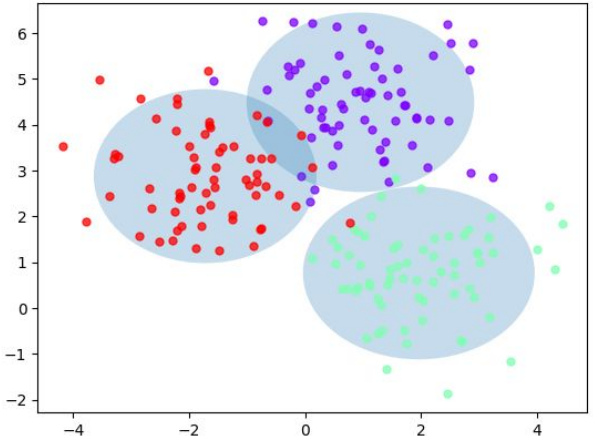


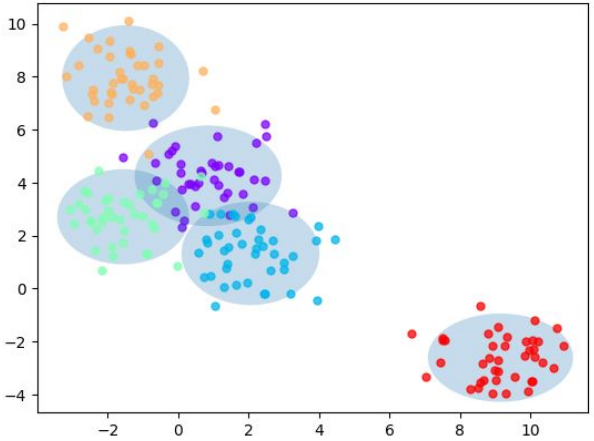


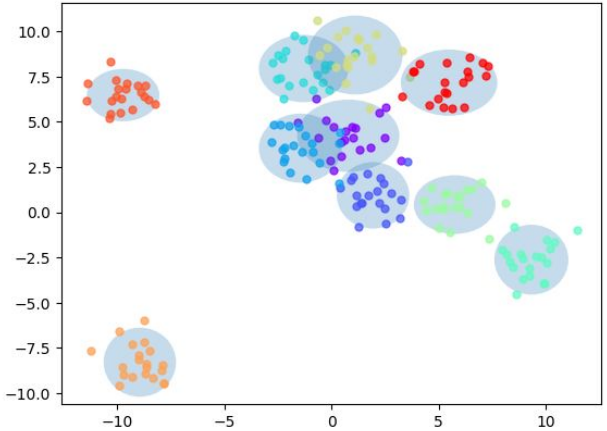


Assignment 1: ML estimates for test data

- The different colors of the scattered dots represent the distinct classes generated. We varied the centers parameter in genBlobs(). Therefore each image displays a different number of classes, specifically showing 3, 5, and 10 separate clusters across your three figures.
- The translucent circular/elliptical shapes represent the 95% confidence interval for each class's Gaussian distribution, based on our computed means and covariances. The center of each ellipse sits exactly at the calculated mean for that class.
- None of the ellipses are tilted; they are all perfectly aligned with the x and y axes. This visual behavior is a direct result of our Naive Bayes assumption. By forcing the off-diagonal elements of the covariance matrix to be zero, the model assumes zero correlation between the x and y axes, restricting the confidence intervals to axis-aligned ellipses.







Assignment 3: testing the classifier

Testing the Bayes Classifier on the Iris dataset we obtained the following two results.

```
testClassifier(BayesClassifier(), dataset='iris', split=0.50)
✓ 0.0s

Trial: 0 Accuracy 89.3
Trial: 10 Accuracy 94.7
Trial: 20 Accuracy 93.3
Trial: 30 Accuracy 88
Trial: 40 Accuracy 85.3
Trial: 50 Accuracy 89.3
Trial: 60 Accuracy 82.7
Trial: 70 Accuracy 89.3
Trial: 80 Accuracy 85.3
Trial: 90 Accuracy 92
Final mean classification accuracy  89.4 with standard deviation 3.11
```

```
testClassifier(BayesClassifier(), dataset='iris', split=0.70)
✓ 0.0s

Trial: 0 Accuracy 84.4
Trial: 10 Accuracy 95.6
Trial: 20 Accuracy 93.3
Trial: 30 Accuracy 86.7
Trial: 40 Accuracy 88.9
Trial: 50 Accuracy 91.1
Trial: 60 Accuracy 86.7
Trial: 70 Accuracy 91.1
Trial: 80 Accuracy 86.7
Trial: 90 Accuracy 91.1
Final mean classification accuracy  89 with standard deviation 4.16
```

These two iterations show a good classification result; moreover including more data into the training dataset does not result into greater accuracy or reduced standard deviation. Instead, the model seems to correctly understand the features of ‘iris’ with just 50% of the total data available given as training dataset

Vowel dataset

```
testClassifier(BayesClassifier(), dataset='vowel', split=0.50)
✓ 0.0s

Trial: 0 Accuracy 60.2
Trial: 10 Accuracy 63.3
Trial: 20 Accuracy 68.6
Trial: 30 Accuracy 57.6
Trial: 40 Accuracy 60.2
Trial: 50 Accuracy 62.5
Trial: 60 Accuracy 64.4
Trial: 70 Accuracy 63.6
Trial: 80 Accuracy 68.2
Trial: 90 Accuracy 67.8
Final mean classification accuracy  63.4 with standard deviation 3.66
```

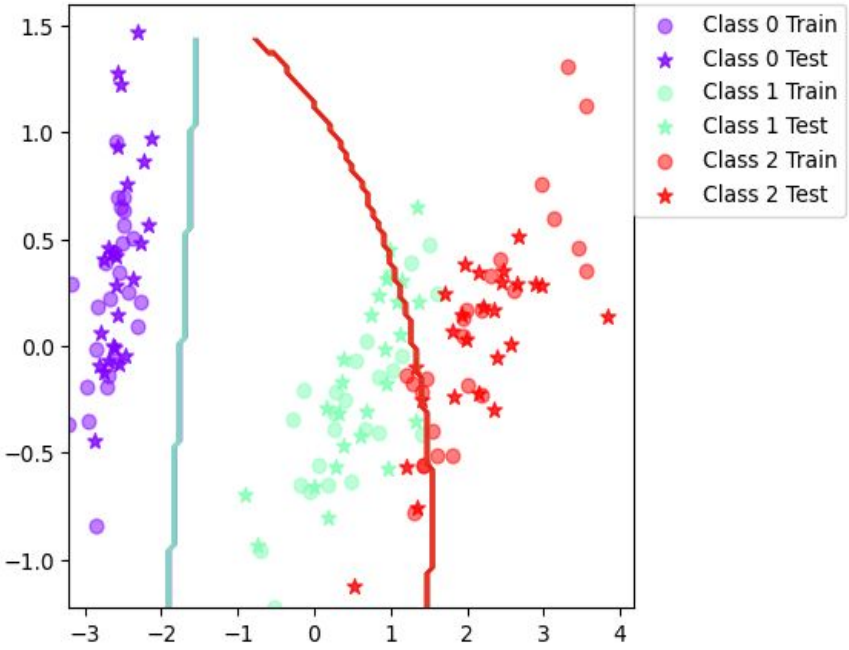
```
testClassifier(BayesClassifier(), dataset='vowel', split=0.70)
✓ 0.0s

Trial: 0 Accuracy 61
Trial: 10 Accuracy 66.2
Trial: 20 Accuracy 74
Trial: 30 Accuracy 66.9
Trial: 40 Accuracy 59.7
Trial: 50 Accuracy 64.3
Trial: 60 Accuracy 66.9
Trial: 70 Accuracy 63.6
Trial: 80 Accuracy 62.3
Trial: 90 Accuracy 70.8
Final mean classification accuracy  64.7 with standard deviation 4.03
```

For this dataset the accuracy is reduced to a less acceptable level. In this case giving more data to the classifier in the training phase shows better performances. This implies that the model is not overfitting and the amount of data available is probably not enough to give the model the ability to memorize all the necessary features and relations among them to correctly classify the data at testing time.

Iris decision boundary

This plot shows the decision boundary between the three classes on the Iris dataset. It looks like a mix of piecewise linear curves or slightly curved hyperplanes.
How to improve it?

1. use a **Full Bayes Classifier**. By calculating the full covariance matrix, the model can "tilt" the decision boundaries to better fit the correlation between features.
2. use **boosting** to combine multiple linear boundaries
3. use **Principal Component Analysis (PCA)** to manipulate the data and show features independence in a new coordinate system where they are really independent



Features independence

The Naive Bayes assumption says that all features are independent with respect to each other given the class label. This eases a lot the procedure of the classifier since the covariance matrix becomes “simply” a diagonal matrix. It works well when the features do not have strong correlations. For example, in text classification (like spam filters), treating the presence of the word "money" as independent of the word "free" is often "good enough" to get high accuracy

Assignment 5: iris dataset

```
testClassifier(BoostClassifier(BayesClassifier(), T=10), dataset='iris',split=0.7)
[24]  ✓ 0.3s

Trial: 0 Accuracy 95.6
Trial: 10 Accuracy 100
Trial: 20 Accuracy 93.3
Trial: 30 Accuracy 91.1
Trial: 40 Accuracy 97.8
Trial: 50 Accuracy 93.3
Trial: 60 Accuracy 93.3
Trial: 70 Accuracy 97.8
Trial: 80 Accuracy 95.6
Trial: 90 Accuracy 93.3
Final mean classification accuracy  94.7 with standard deviation 2.82
```

Comparison between the basic and boosted classifier on the iris dataset

	Basic classifier	Boosted Classifier
mean accuracy	89.0%	94.7%
standard deviation	4.16	2.82

There is a distinct improvement in classification accuracy for both datasets. This is because boosting generates a composite classifier with a smaller error on the training data by aggregating multiple hypotheses. It works by repeatedly invoking the weak learning algorithm and changing the distribution of the training instances after each iteration based on the classifier's error. The weights of incorrectly classified instances are increased, meaning the next iteration of the classifier will focus heavily on correcting the mistakes of the previous one. Because our base Naive Bayes classifier is relatively weak, grouping many of them together and forcing them to specialize in hard-to-classify points yields a much stronger overall model.

Assignment 5: vowel dataset

```
testClassifier(BoostClassifier(BayesClassifier(), T=10), dataset='vowel',split=0.7)
[25]  ✓ 1.0s

Trial: 0 Accuracy 76.6
Trial: 10 Accuracy 86.4
Trial: 20 Accuracy 83.1
Trial: 30 Accuracy 80.5
Trial: 40 Accuracy 72.7
Trial: 50 Accuracy 76
Trial: 60 Accuracy 81.8
Trial: 70 Accuracy 82.5
Trial: 80 Accuracy 79.9
Trial: 90 Accuracy 83.1
Final mean classification accuracy  80.2 with standard deviation 3.52
```

Comparison between the basic and boosted classifier on the vowel dataset

	Basic classifier	Boosted Classifier
mean accuracy	64.7%	80.2%
standard deviation	4.03	3.52

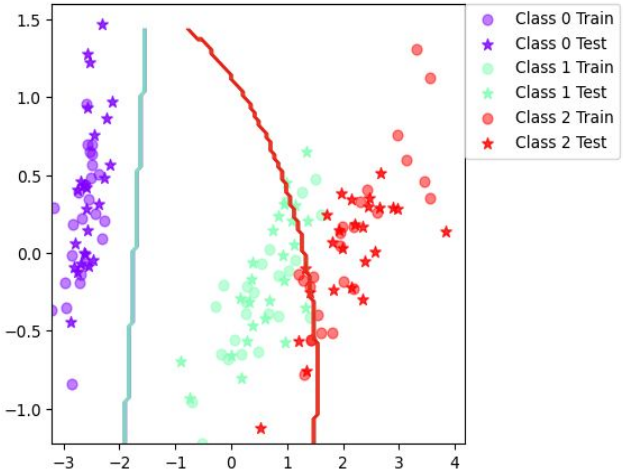
Boosting resulted in a massive improvement of over 15 percentage points and, again, slightly reduced the variance.

Assignment 5: decision boundaries

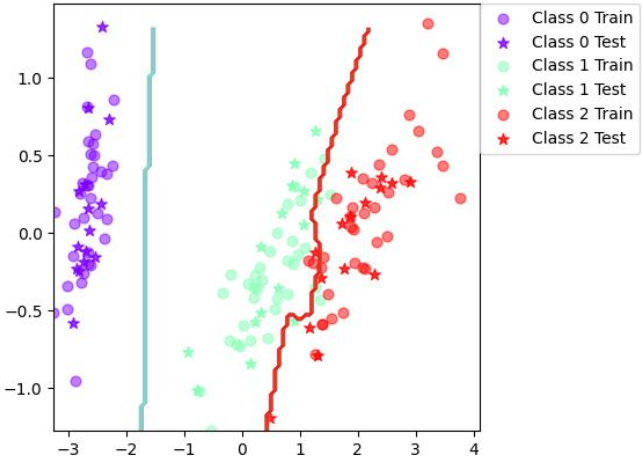
In the image on the left, the decision boundary between Class 1 (green) and Class 2 (red) is a smooth, continuous curve. This is characteristic of a standard Gaussian Naive Bayes classifier, where the boundary is defined by quadratic equations derived from a single set of means and variances.

In the image on the right, the boundary produced by the boosted classifier between Class 1 and Class 2 is much more jagged and piecewise.

Basic classifier



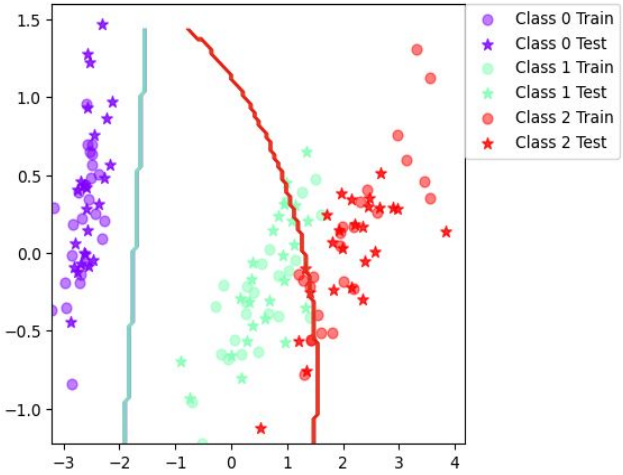
Boosted classifier



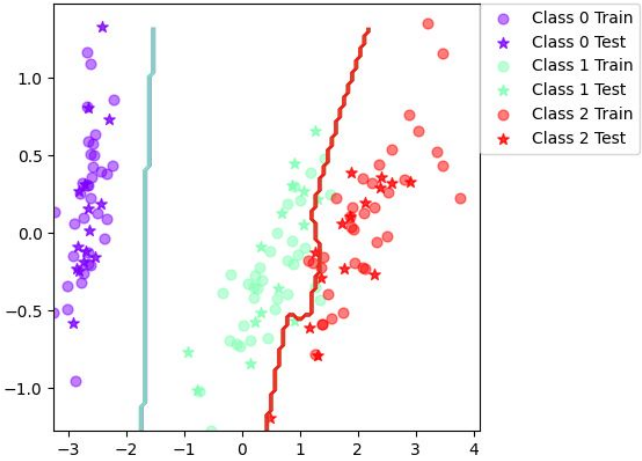
Assignment 5: decision boundaries

Therefore, the boosted version is noticeably more complex. Because the boosted classifier aggregates the votes of T different base classifiers, each trained on a uniquely weighted distribution of the data, the resulting composite boundary is a combination of many simpler boundaries. This allows it to wrap more tightly around the specific distributions of the training points.

Basic classifier



Boosted classifier

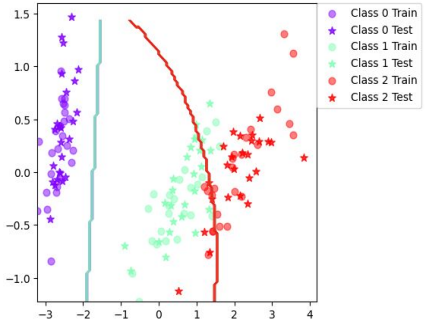


Assignment 5: decision boundaries

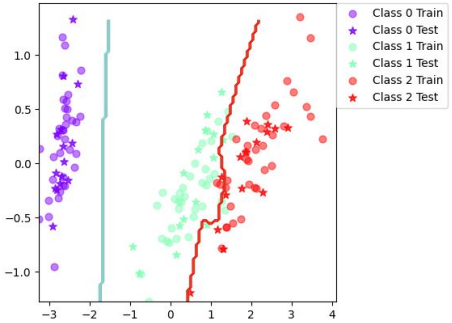
Can we make up for not using a more advanced model in the basic classifier (e.g. independent features) by using boosting?

Yes, boosting effectively compensates for the simplicity of the base model. Our basic classifier assumes that all feature dimensions are uncorrelated (the Naive Bayes assumption), which simplifies the covariance matrix to only its diagonal elements. This assumption strictly limits the model; for instance, it can only draw axis-aligned confidence intervals and relatively simple decision boundaries. However, by applying the Adaboost algorithm, we aggregate multiple of these simple models. While each individual hypothesis is constrained by the independence assumption, their combined, weighted output can approximate highly complex, non-linear relationships. The dramatic jump in accuracy on the Vowel dataset proves that boosting can make a simple model highly competitive.

Basic classifier



Boosted classifier



Assignment 6: iris dataset

```
testClassifier(DecisionTreeClassifier(), dataset='iris', split=0.5)
✓ 0.0s

Trial: 0 Accuracy 96
Trial: 10 Accuracy 94.7
Trial: 20 Accuracy 94.7
Trial: 30 Accuracy 89.3
Trial: 40 Accuracy 92
Trial: 50 Accuracy 93.3
Trial: 60 Accuracy 89.3
Trial: 70 Accuracy 93.3
Trial: 80 Accuracy 85.3
Trial: 90 Accuracy 90.7
Final mean classification accuracy  92.6 with standard deviation 2.34
```

```
testClassifier(BoostClassifier(DecisionTreeClassifier(), T=10), dataset='iris',split=0.5)
✓ 0.3s

Trial: 0 Accuracy 90.7
Trial: 10 Accuracy 96
Trial: 20 Accuracy 93.3
Trial: 30 Accuracy 90.7
Trial: 40 Accuracy 94.7
Trial: 50 Accuracy 93.3
Trial: 60 Accuracy 90.7
Trial: 70 Accuracy 94.7
Trial: 80 Accuracy 92
Trial: 90 Accuracy 92
Final mean classification accuracy  93.6 with standard deviation 2.77
```

These two pictures show the impact of boosting on the decision tree. Clearly there is an increase in terms of accuracy using boosting with 10 classifiers. We have, anyways, an increase also in the standard deviation, which indicates a bit more variance, that is not what we expect normally with boosting. We also need to consider that the single decision tree classifier is already very precise and models very well the iris dataset.

Vowel dataset

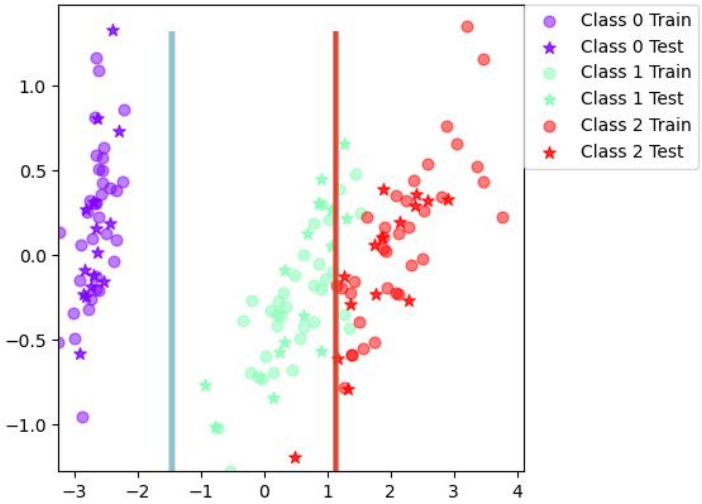
```
testClassifier(DecisionTreeClassifier(), dataset='vowel',split=0.70)
✓ 0.1s

Trial: 0 Accuracy 63.6
Trial: 10 Accuracy 68.8
Trial: 20 Accuracy 63.6
Trial: 30 Accuracy 66.9
Trial: 40 Accuracy 59.7
Trial: 50 Accuracy 63
Trial: 60 Accuracy 59.7
Trial: 70 Accuracy 68.8
Trial: 80 Accuracy 59.7
Trial: 90 Accuracy 68.2
Final mean classification accuracy  64.1 with standard deviation 4.02
```

```
testClassifier(BoostClassifier(DecisionTreeClassifier(), T=10), dataset='vowel',split=0.7)
✓ 1.5s

Trial: 0 Accuracy 85.7
Trial: 10 Accuracy 90.3
Trial: 20 Accuracy 87.7
Trial: 30 Accuracy 93.5
Trial: 40 Accuracy 83.1
Trial: 50 Accuracy 79.9
Trial: 60 Accuracy 90.9
Trial: 70 Accuracy 85.7
Trial: 80 Accuracy 83.1
Trial: 90 Accuracy 85.1
Final mean classification accuracy  86.6 with standard deviation 3.21
```

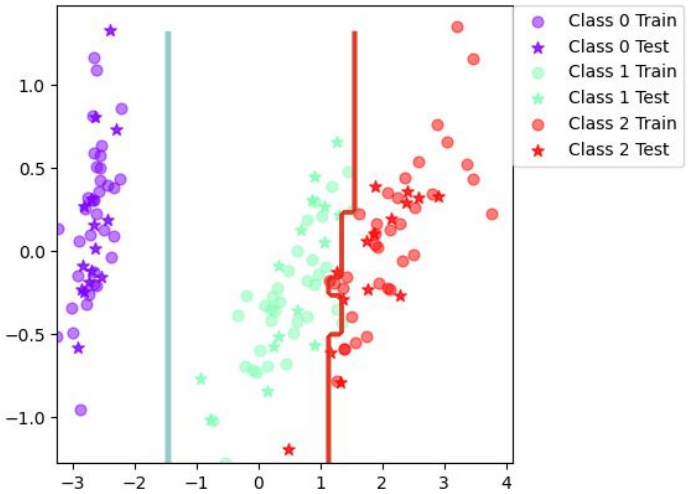
With this dataset the impact of boosting is way clearer. The accuracy increases dramatically and also the standard deviation decreases. Given that the initial model was not so precise in the classification, we have obtained great results implementing boosting. In this case boosting will absolutely make up for the use of a more advanced model as classifier.

Iris decision boundary

Without boosting



Boosted



The decision boundary is less complex with respect to the one of a Bayes Classifier. Again. introducing boosting, implies a complex and more precise decision boundary. The performances shows before are a proof that this increasing complexity (in fact only for the boundary between class 1 and 2) is effective in terms of accuracy and it is not a sign of overfitting and limited generalization.

Assignment 7: analysis of the models

Outliers:

1. *Naive bayes*: **relatively robust** since it uses mean and variance of the entire class distributions
2. *Decision Tree*: highly **sensitive** to them
3. *Boosted NV/ boosted DT:* we can **worsen the impact of outliers** since the algorithm will repetitively increase the weights of those points “hard to classify”

Irrelevant inputs:

1. *Naive Bayes*: performance **degrades** a lot if many of them are present
2. *Decision Tree*: the tree **naturally avoids the most irrelevant features** when deciding the attributes used to split
3. *Boosted NV/ boosted DT*: it **works okay** and can help consider them not so much

Predictive power:

1. *Naive Bayes*: **good for simple datasets**, **limited by the Bayes assumption**
2. *Decision tree*: **precise** but prone to **high variance**
3. *Boosted NV/Boosted DT*: they offer the **highest predictive power** as shown before

Other metrics

Mixed types of data:

1. *Naive Bayes*: **best for continuous or categorical data**; requires some modifications for mixing data
2. *Decision tree*: the **best** in this metric
3. *Boosted NB/Boosted DT*: **inherits the flexibility** of its basic learner

Scalability:

1. *Naive Bayes*: **highly scalable** since it requires only one pass towards the data
2. *Decision tree*: **scalable** even though **training becomes more complex**
3. *Boosted NB/Boosted DT*: **least scalable** due to higher training time needed

So which is the one to choose?

Boosted Decision Trees if **accuracy** is the top priority and you have **mixed data types or irrelevant features**.

Naive Bayes when speed and simplicity are required. Very effective also for very **high-dimensional data** (like 10,000+ features) where **features are mostly independent.**